

// Jivops

Guía completa — 30 lecciones

Testing · Arquitectura · Concurrency · APIs · Empaquetado ·
Observabilidad

Contenido

Módulo 1: Código de calidad y testing

- Lección 01 — Type hints y mypy
- Lección 02 — Testing con pytest — fundamentos
- Lección 03 — Fixtures y mocking
- Lección 04 — Cobertura de tests y CI
- Lección 05 — Logging estructurado

Módulo 2: Arquitectura y patrones

- Lección 06 — Manejo de excepciones en producción
- Lección 07 — Context managers avanzados
- Lección 08 — Decoradores en producción
- Lección 09 — Dataclasses y Pydantic
- Lección 10 — Arquitectura en capas (services/repos)

Módulo 3: Concurrencia y rendimiento

- Lección 11 — Concurrencia con threading
- Lección 12 — Concurrencia con multiprocessing
- Lección 13 — Programación asíncrona con asyncio
- Lección 14 — async/await en la práctica
- Lección 15 — Rendimiento: profiling con cProfile

Módulo 4: APIs y servicios

- Lección 16 — Construir una API con FastAPI
- Lección 17 — Validación de datos con Pydantic
- Lección 18 — Autenticación JWT en FastAPI
- Lección 19 — Rate limiting y throttling
- Lección 20 — Documentación automática de APIs (OpenAPI)

Módulo 5: Empaquetado y despliegue

- Lección 21 — Empaquetado con pyproject.toml
- Lección 22 — Entornos virtuales y dependencias reproducibles
- Lección 23 — Dockerizar una app Python
- Lección 24 — Variables de entorno y configuración segura (12-factor)
- Lección 25 — CI/CD para proyectos Python

Módulo 6: Observabilidad y seguridad en producción

Lección 26 — Logging centralizado y correlación de requests

Lección 27 — Métricas con Prometheus client

Lección 28 — Manejo de secretos (Vault, env vars seguras)

Lección 29 — Seguridad: dependencias vulnerables (pip-audit)

Lección 30 — Proyecto final: API Python lista para producción

Módulo 1: Código de calidad y testing

Lección 01 — Type hints y mypy

Las type hints declaran el tipo esperado de variables, parámetros y retornos. mypy analiza el código de forma estática y detecta inconsistencias de tipo antes de ejecutar, sin cambiar el comportamiento en runtime de Python.

```
def suma(a: int, b: int) -> int:
    return a + b
```

Lección 02 — Testing con pytest — fundamentos

pytest descubre automáticamente funciones que empiezan con test_ y usa `assert` simple para las comprobaciones, sin necesidad de clases ni métodos especiales como en unittest.

```
def test_suma():
    assert suma(2, 3) == 5
```

Lección 03 — Fixtures y mocking

Las fixtures preparan datos o recursos reutilizables antes de cada test. El mocking sustituye dependencias externas (APIs, bases de datos) por objetos simulados, haciendo los tests rápidos y aislados.

```
@pytest.fixture
def usuario():
    return {"id": 1, "nombre": "Ana"}
```

Lección 04 — Cobertura de tests y CI

La cobertura mide qué porcentaje del código se ejecuta durante los tests. Es una guía útil, pero no garantiza ausencia de bugs — solo indica qué líneas nunca se probaron.

```
pytest --cov=app --cov-report=term-missing
```

Lección 05 — Logging estructurado

El logging en formato JSON (estructurado) permite buscar y filtrar logs en un sistema centralizado por campos concretos, en vez de hacer grep sobre texto plano poco predecible.

```
logger = logging.getLogger(__name__)
logger.info("pago procesado", extra={"request_id": rid})
```

Módulo 2: Arquitectura y patrones

Lección 06 — Manejo de excepciones en producción

Capturar `except Exception` sin distinción oculta errores inesperados y dificulta el debugging. Es mejor definir excepciones específicas y personalizadas por dominio (ej. `PagoInvalidoError`).

```
class PagoInvalidoError(Exception):  
    pass
```

Lección 07 — Context managers avanzados

Un context manager (`with`) garantiza que un recurso se libere correctamente incluso si ocurre una excepción, implementando los métodos `__enter__` y `__exit__`.

Lección 08 — Decoradores en producción

Los decoradores añaden comportamiento (logging, reintentos, medición de tiempo) a una función sin modificar su código interno. `functools.wraps` preserva el nombre y docstring originales.

```
@functools.wraps(func)  
def wrapper(*args, **kwargs):  
    return func(*args, **kwargs)
```

Lección 09 — Dataclasses y Pydantic

Una dataclass estructura datos simples sin validación. Pydantic añade validación y conversión automática de tipos al crear la instancia, ideal para datos que entran desde fuera del sistema.

Lección 10 — Arquitectura en capas (services/repos)

Separar el acceso a datos (repository) de la lógica de negocio (service) permite testear la lógica con mocks, sin depender de una base de datos real en cada test.

Módulo 3: Concurrency y rendimiento

Lección 11 — Concurrency con threading

El GIL (Global Interpreter Lock) limita el paralelismo real de CPU con threads en Python. Aun así, threading es útil para tareas I/O-bound como peticiones de red, donde el hilo espera sin ocupar CPU.

Lección 12 — Concurrency con multiprocessing

multiprocessing evita el GIL usando procesos separados, cada uno con su propio intérprete. Es la opción correcta para tareas CPU-bound como cálculos intensivos.

```
from multiprocessing import Pool
with Pool(4) as p:
    p.map(procesar, datos)
```

Lección 13 — Programación asíncrona con asyncio

asyncio permite manejar muchas tareas I/O concurrentes en un solo hilo, atendiendo otras tareas mientras una espera respuesta de red, sin bloquear el programa.

Lección 14 — async/await en la práctica

Llamar a una función `async def` sin `await` solo crea un objeto coroutine, no ejecuta su código. Mezclar código bloqueante (ej. `time.sleep()`) dentro de una función async bloquea todo el event loop.

```
async def obtener_datos():
    datos = await fetch(url)
    return datos
```

Lección 15 — Rendimiento: profiling con cProfile

Antes de optimizar código "a ciegas", se perfila con cProfile para identificar el cuello de botella real — la función donde realmente se pasa más tiempo.

```
python -m cProfile -s cumulative script.py
```

Módulo 4: APIs y servicios

Lección 16 — Construir una API con FastAPI

FastAPI genera documentación interactiva automática (OpenAPI/Swagger) a partir de las rutas y modelos definidos, usando Pydantic para validar los datos de entrada.

```
@app.get("/items")
async def listar_items():
    return items
```

Lección 17 — Validación de datos con Pydantic

Si los datos enviados no cumplen el esquema definido en un modelo Pydantic, FastAPI devuelve automáticamente un error 422 con el detalle, sin necesidad de validar manualmente con condicionales.

Lección 18 — Autenticación JWT en FastAPI

Un JWT contiene datos del usuario firmados digitalmente, permitiendo verificar su identidad sin consultar la base de datos en cada petición. Debe tener fecha de expiración para limitar el riesgo si es robado.

```
Authorization: Bearer <token>
```

Lección 19 — Rate limiting y throttling

El rate limiting evita que un cliente sature el servicio con demasiadas peticiones. Cuando se excede el límite, la API suele responder con el código 429 Too Many Requests.

Lección 20 — Documentación automática de APIs (OpenAPI)

La documentación en /docs se genera directamente del código, por lo que nunca queda desactualizada respecto a las rutas y modelos reales de la API.

Módulo 5: Empaquetado y despliegue

Lección 21 — Empaquetado con pyproject.toml

pyproject.toml unifica en un solo archivo la configuración que antes vivía dispersa entre setup.py y requirements.txt, centralizando el empaquetado del proyecto.

```
[build-system]
requires = ["setuptools"]
```

Lección 22 — Entornos virtuales y dependencias reproducibles

Fijar versiones exactas de dependencias (ej. requests==2.31.0) garantiza que el entorno se comporte igual en todos los despliegues, evitando roturas por actualizaciones silenciosas.

```
pip freeze > requirements.txt
```

Lección 23 — Dockerizar una app Python

Usar una imagen base slim o alpine reduce el tamaño de la imagen y la superficie de ataque. Un build multi-stage deja la imagen final sin herramientas de compilación innecesarias.

```
FROM python:3.12-slim
WORKDIR /app
COPY . .
RUN pip install -r requirements.txt
```

Lección 24 — Variables de entorno y configuración segura (12-factor)

Según la metodología 12-factor, credenciales y configuración sensible deben vivir en variables de entorno, nunca hardcodeadas ni versionadas en el código fuente.

```
os.environ.get("DATABASE_URL")
```

Lección 25 — CI/CD para proyectos Python

Un pipeline típico instala dependencias, corre tests y lint, construye la imagen y despliega — automatizando validaciones que, hechas a mano, se olvidan con el tiempo.

Módulo 6: Observabilidad y seguridad en producción

Lección 26 — Logging centralizado y correlación de requests

Un identificador único (`request_id`) propagado entre servicios permite correlacionar los logs de una misma petición aunque atravesase varios microservicios distintos.

```
X-Request-ID: 7f3a2c1e
```

Lección 27 — Métricas con Prometheus client

Un Counter mide valores que solo aumentan (peticiones totales); un Gauge mide valores que suben y bajan libremente (conexiones activas). Elegir el tipo correcto importa para interpretar bien el dato.

```
from prometheus_client import Counter
peticiones = Counter('peticiones_total', 'desc')
```

Lección 28 — Manejo de secretos (Vault, env vars seguras)

Un gestor de secretos como Vault aporta rotación, auditoría y control de acceso granular — algo que las variables de entorno planas no ofrecen por sí solas.

Lección 29 — Seguridad: dependencias vulnerables (pip-audit)

pip-audit revisa las dependencias instaladas contra bases de datos de vulnerabilidades conocidas. Debe correrse periódicamente, ya que se descubren nuevas vulnerabilidades constantemente.

```
pip-audit
```

Lección 30 — Proyecto final: API Python lista para producción

Se integra todo lo anterior: una API con FastAPI, tests con pytest, Dockerizada, con CI/CD, logging estructurado, métricas en Prometheus y gestión segura de secretos. Este es el nivel esperado de un desarrollador Python en producción real.